Lecture 7 — Advanced Combinatorics: Burnside/Pólya, Möbius Inversion, Generating Functions, Sublinear Sums

> World Finals Level · Prerequisites: Year weeks 10–11 and 30 (number theory, combinatorics), regional-07 (φ, Möbius intro), worldfinals-02 (NTT, polynomial inverse/log/exp) · Estimated time: 2 × 90 min lecture + 3 h lab

1. Overview

Counting problems at the World Finals and in Codeforces Div 1 E/F rarely need a single formula. They need a *pipeline*: a symmetry argument reduces the problem to counting fixed objects; a Möbius inversion turns a gcd condition into a sum over divisors; a generating function turns a recursive structure into a polynomial equation; and a sublinear sieve evaluates the resulting sum over n up to 10^{11}. This lecture covers each stage:

1. **Burnside's lemma and Pólya enumeration** — counting up to symmetry, including with constraints.
2. **Dirichlet convolution and Möbius inversion** — the algebra of divisor sums, and floor-division blocks.
3. **Generating functions** — the combinatorial dictionary (sum, product, sequence, set, multiset) and when to hand the result to the polynomial toolkit from Lecture 2.
4. **Sublinear multiplicative sums** — Du's sieve, Lucy's prime-counting DP, and the min-25 sieve.

All code below is tested against brute force (orbit enumeration for Burnside; a linear sieve up to $2 \cdot 10^6$ for the sublinear sums).

2. Burnside's lemma

2.1 Statement and proof

Let a finite group G act on a finite set X. For $g \in G$ let $\mathrm{Fix}(g) = \{ x \in X : g \cdot x = x \}$. Then

> number of orbits = $(1/|G|) \cdot \Sigma_{g \in G} |\mathrm{Fix}(g)|$.

Proof. Count pairs (g, x) with g·x = x in two ways. Summing over g gives Σ_g |Fix(g)|. Summing over x gives Σ_x |Stab(x)|, and by the orbit–stabiliser theorem |Stab(x)| = |G| / |Orbit(x)|. The elements of one orbit O contribute |O| · |G|/|O| = |G| in total, so the sum is |G| times the number of orbits. ∎

The lemma converts "count equivalence classes of unequal sizes" (hard) into "for each symmetry, count the objects it does not change" (usually easy, because an object fixed by g must be constant on each cycle of g).

2.2 Necklaces

X = colourings of n beads with k colours, G = the n rotations. The rotation by i positions decomposes the beads into gcd(i, n) cycles, each of length n / gcd(i, n). A colouring is fixed iff it is constant on every cycle, so |Fix(rot_i)| = k^{gcd(i, n)}. The number of i ∈ [0, n) with gcd(i, n) = n/d is φ(d). Hence

> N(n, k) = (1/n) · Σ_{d | n} φ(d) · k^{n/d}.

With n up to 10^9 there are at most 1344 divisors; computing φ(d) by trial division is fine, or factor n once and generate divisors together with their φ values.

Modular division by n requires n to be invertible modulo the prime. POJ 2154 *Color* asks for the answer modulo an arbitrary (possibly composite) P, but there the number of colours equals the number of beads (k = n), so (1/n) Σ φ(d) n^{n/d} = Σ φ(d) n^{n/d − 1} and no division is needed. Look for such cancellations whenever the modulus is not a large prime.

2.3 Bracelets

Allowing reflections gives the dihedral group of order 2n. For odd n every reflection axis passes through one bead and the opposite edge midpoint: (n + 1)/2 cycles, so k^{(n+1)/2} fixed colourings, n such reflections. For even n, n/2 axes pass through two opposite beads (n/2 + 1 cycles) and n/2 axes pass through two edge midpoints (n/2 cycles). So

> B(n, k) = [n·N(n, k) + R(n, k)] / (2n), with R = n·k^{(n+1)/2} (n odd) or (n/2)(k^{n/2} + k^{n/2+1}) (n even).

2.4 Burnside with constraints

Constraints change only the count of fixed points. Example: necklaces in which adjacent beads have different colours. A rotation with gcd(i, n) = d fixes exactly the colourings with period d, which are determined by beads 0..d−1. The adjacency constraints for the whole necklace become: consecutive beads among the first d differ, and bead d−1 differs from bead 0 (since bead d = bead 0). That is a proper colouring of the cycle C_d, of which there are (k−1)^d + (−1)^d (k−1). (For d = 1 this is 0, correctly: a single bead adjacent to itself.) Therefore

> $(1/n) \cdot \Sigma_{d \mid n} \varphi(n/d) \cdot [\,(k-1)^d + (-1)^d (k-1)\,]$.

In general, if validity is a local condition along the necklace given by a transfer matrix T ($T[a][b] = 1$ if colour b may follow colour a), the number of valid colourings of a cycle of length d is the trace $\operatorname{tr}(T^d)$, and the formula becomes $(1/n) \Sigma_{d \mid n} \varphi(n/d) \operatorname{tr}(T^d)$. This pattern (Burnside + trace of a matrix power) appears in many hard necklace problems.

2.5 Pólya enumeration

The cycle index of G is $Z(G) = (1/|G|) \Sigma_g \Pi_j a_j^{c_j(g)}$, where $c_j(g)$ is the number of cycles of length j in g. Substituting $a_j = x_1^j + \dots + x_k^j$ gives a polynomial whose coefficient of $x_1^{e_1} \cdots x_k^{e_k}$ is the number of orbits of colourings with exactly e_i positions of colour i. The reason is the same as in Burnside: in a fixed colouring each cycle is monochromatic, and a cycle of length j contributes j positions of one colour.

For two colours on a necklace, "exactly r red beads" gives $(1/n) \Sigma_{d \mid \gcd(n, r)} \varphi(d) \cdot C(n/d, r/d)$. For many colours with given multiplicities, each group element's contribution is a multinomial count of ways to distribute cycles of each length among the colours — a small knapsack over cycle lengths.

2.6 Irregular groups

For symmetries of a cube, of a small graph, or of a grid (rotations and reflections of an $n \times n$ board), generate the group explicitly: write each generator as a permutation of the positions (faces, edges, cells), then take the closure under composition with a BFS over permutations. The cube rotation group has 24 elements; the full symmetry group of a square grid has 8. Compute each element's cycle type and apply Burnside or Pólya. For a grid with n^2 cells and the 4 rotations, CSES 2210 *Counting Grids* gives $(k^{n^2} + 2k^{\lceil n^2/4 \rceil} + k^{\lceil n^2/2 \rceil}) / 4$.

3. Dirichlet convolution and Möbius inversion

3.1 The algebra

For arithmetic functions, $(f * g)(n) = \Sigma_{d \mid n} f(d)\, g(n/d)$. This product is commutative and associative, with identity $\varepsilon(n) = [n = 1]$. Write 1 for the constant function, $\mathrm{id}(n) = n$. The key identities:

- $\mu * 1 = \varepsilon$ (definition of μ; equivalently $\Sigma_{d \mid n} \mu(d) = [n = 1]$);
- $\varphi * 1 = \mathrm{id}$, hence $\varphi = \mu * \mathrm{id}$;
- $1 * 1 = d$ (number of divisors), $\mathrm{id} * 1 = \sigma$ (sum of divisors).

Möbius inversion: $g = f * 1 \Leftrightarrow f = g * \mu$. The convolution of multiplicative functions is multiplicative, which is what makes sieves work.

3.2 Turning gcd conditions into sums

The workhorse identity is [gcd(i, j) = 1] = Σ_{d | gcd(i, j)} μ(d). Swapping the order of summation:

> Σ_{i ≤ n} Σ_{j ≤ m} [gcd(i, j) = 1] = Σ_{d ≥ 1} μ(d) ⌊n/d⌋ ⌊m/d⌋.

Similarly gcd(i, j) = Σ_{d | gcd} φ(d) gives Σ gcd(i, j) = Σ_d φ(d) ⌊n/d⌋ ⌊m/d⌋. More elaborate sums (lcm sums, Σ f(gcd)) follow the same steps: express the summand as a divisor sum, swap, and count multiples.

The same trick works on *sets*: "number of subsequences with gcd 1" (CF 803F) = Σ_d μ(d) (2^{c_d} − 1), where c_d is the number of elements divisible by d. "Number of coprime pairs in a dynamic multiset" (CF 547C) maintains c_d over the square-free divisors of each inserted value.

3.3 Floor-division blocks

⌊n/d⌋ takes at most 2√n distinct values, and for a given l the largest r with ⌊n/r⌋ = ⌊n/l⌋ is r = ⌊n / ⌊n/l⌋⌋. With two variables take the minimum of both bounds. So any sum Σ_d g(d) · F(⌊n/d⌋, ⌊m/d⌋) costs O(√n + √m) evaluations given prefix sums of g. For n ≤ 10^7 the prefix sums of μ or φ come from a linear sieve; beyond that you need the sublinear methods of §5.

4. Generating functions

4.1 The dictionary

For a class of objects counted by size, the ordinary generating function (OGF) is A(x) = Σ a_n x^n; for labelled objects use the exponential generating function (EGF) Σ a_n $x^n/n!$. Constructions map to operations:

Construction	Unlabelled (OGF)	Labelled (EGF)
disjoint union	A + B	A + B
ordered pair	A · B	A · B (relabel: binomial convolution)
sequence	1 / (1 − A)	1 / (1 − A)
multiset / set	exp(Σ_{k≥1} A(x^k)/k)	exp(A)
cycle	Σ_{k≥1} φ(k)/k · log(1/(1 − A(x^k)))	log(1/(1 − A))

The unlabelled cycle construction is Burnside again, applied to sequences under rotation — the two halves of this lecture meet here.

4.2 Classic examples

- **Partitions.** $\prod_{k \geq 1} 1/(1 - x^k)$. Euler's pentagonal theorem says $\prod (1 - x^k) = \sum_{k \in \mathbb{Z}} (-1)^k x^{k(3k-1)/2}$, so $p(n) = \sum_{k \geq 1} (-1)^{k+1} [p(n - k(3k-1)/2) + p(n - k(3k+1)/2)]$, an $O(n\sqrt{n})$ recurrence (§6.3), or a single power-series inverse in $O(n \log n)$.
- **Connected labelled graphs.** All graphs: $G(x) = \sum 2^{C(n,2)} x^n/n!$. Every graph is a set of connected components, so $G = \exp(C)$ and $C = \log G$.
- **Binary trees with vertex weights from a set S** (CF 438E). With $S(x) = \sum_{s \in S} x^s$, $F = 1 + S \cdot F^2$, so $F = 2 / (1 + \sqrt{(1 - 4S)})$ — one polynomial square root and one inverse.
- **Rational generating functions** are exactly linear recurrences; Berlekamp–Massey (Lecture 2) recovers the recurrence from the first $2L$ terms, which is often the fastest route when you can brute-force small cases.

4.3 Workflow

When a counting problem has a recursive structure, write the GF equation first. Then decide: is n small enough for an $O(n^2)$ DP? Does the equation have a closed form? Is it a rational function (then a linear recurrence and Kitamasa / matrix power)? Or do you need $O(n \log n)$ exp/log/inverse/sqrt (Lecture 2)? Choosing the representation before writing code avoids many hours of DP debugging.

5. Sublinear sums of multiplicative functions

All methods here work on the set $V = \{ \lfloor n/i \rfloor : 1 \leq i \leq n \}$, which has at most $2\sqrt{n}$ elements. Index it with two arrays: values $v \leq \sqrt{n}$ by v itself, larger values by n / v. The code below stores V in descending order and maps $v \mapsto$ index in $O(1)$.

5.1 Du's sieve

Suppose we want $F(n) = \sum_{i \leq n} f(i)$, and we know an arithmetic function g with $g(1) = 1$ such that both G (prefix sums of g) and H (prefix sums of $h = f * g$) are easy to evaluate. Then

$$H(n) = \sum_{i \leq n} \sum_{d \mid i} g(d) f(i/d) = \sum_{d \leq n} g(d) F(\lfloor n/d \rfloor),$$

so $F(n) = H(n) - \sum_{d=2}^{n} g(d) F(\lfloor n/d \rfloor)$. Every argument $\lfloor n/d \rfloor$ lies in V, the inner sum is evaluated in floor blocks, and results are memoised. If F is precomputed by a linear sieve up to $n^{2/3}$, the total cost is $O(n^{2/3})$.

- $f = \varphi$, $g = 1$, $h = \mathrm{id}$: $\Phi(n) = n(n+1)/2 - \sum_{d \geq 2} \Phi(\lfloor n/d \rfloor)$.
- $f = \mu$, $g = 1$, $h = \varepsilon$: $M(n) = 1 - \sum_{d \geq 2} M(\lfloor n/d \rfloor)$.
- $f = \mu \cdot \mathrm{id}$ (i.e. $\mu(i) \cdot i$), $g = \mathrm{id}$, $h = \varepsilon$: same pattern with weights d.

5.2 Lucy's prime-counting DP (min-25 part 1)

Define, after processing primes up to p, g_p(v) = number of integers in [2, v] that are prime or have smallest prime factor > p. Initially (p = 1) g(v) = v − 1. Processing prime p removes the integers whose smallest prime factor is exactly p; those are p·m with m ∈ [p, v/p] having no prime factor < p, and their count is g(⌊v/p⌋) − g(p − 1) (the subtraction removes the primes smaller than p, which are counted in g(⌊v/p⌋) but are not valid m). This is only needed when p^2 ≤ v. After all primes up to √n, g(v) = π(v) for every v ∈ V simultaneously.

Replacing "count" by "sum of i^k" (initial value Σ_{i=2}^{v} i^k, update multiplied by p^k) gives prime power sums Σ_{p ≤ v} p^k. The complexity is O(n^{3/4} / log n); $\pi(10^{10})$ takes a few hundredths of a second, $\pi(10^{11})$ well under a second.

5.3 min-25 part 2

Let f be multiplicative with f(p) given by a polynomial in p (so part 1 gives G(v) = Σ_{p ≤ v} f(p) for v ∈ V) and f(p^e) computable in O(1). Let $p_0 < p_1 < \ldots$ be the primes up to √n and define S(x, j) = Σ f(i) over 2 ≤ i ≤ x whose smallest prime factor is ≥ p_j. Splitting by the smallest prime factor p_k and its exponent e:

S(x, j) = [G(x) − G(p_j − 1)] + Σ_{k ≥ j, p_k^2 ≤ x} Σ_{e ≥ 1, p_k^{e+1} ≤ x} [f(p_k^e) · S(⌊x/p_k^e⌋, k + 1) + f(p_k^{e+1})].

The first bracket counts primes ≥ p_j; the inner sum counts composites p_k^e · m with m > 1 having all prime factors > p_k (the recursive term) and the prime powers p_k^{e+1} themselves. The answer is S(n, 0) + f(1). The recursion is evaluated without memoisation; its node count is about n^{3/4}/log n in practice, so n = 10^{10} takes ~0.3 s and n = 10^{11} ~1–2 s with tight code.

5.4 Which method?

- **Du's sieve** when a convenient Dirichlet partner g exists (φ, μ, and their products with id^k). It is the shortest code.
- **Lucy** for anything about primes only: π(n), Σ p, counting numbers of the form p·q or p^3 (CF 665F *Four Divisors*: n = p^3 or n = p·q with p < q, i.e. Σ_p [π(n/p) − π(p)] over p ≤ √n plus cubes).
- **min-25** for general multiplicative f whose values at prime powers are irregular (σ_k, f(p^e) = p xor e as in LOJ 6053).

6. Reference implementations

6.1 Burnside: necklaces, bracelets, proper necklaces

Input: n k ($n \le 10^{12}$, $k \le 10^{18}$). Output, modulo $10^9 + 7$: the number of necklaces, bracelets, and necklaces with adjacent beads distinct.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 1'000'000'007;

ll pw(ll b, ll e) {
    b %= MOD; if (b < 0) b += MOD;
    ll r = 1;
    for (; e; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
    return r;
}
ll inv(ll a) { return pw(a, MOD - 2); }

// divisors of n together with phi(d), from the factorisation of n
vector<pair<ll, ll>> divisorsWithPhi(ll n) {
    vector<pair<ll, int>> f;
    for (ll p = 2; p * p <= n; p++) if (n % p == 0) { int e = 0; while (n % p == 0) n /=
p, e++; f.push_back({p, e}); }
    if (n > 1) f.push_back({n, 1});
    vector<pair<ll, ll>> res = {{1, 1}};
    for (auto [p, e] : f) {
        int sz = res.size();
        ll pk = 1, ph = 1;
        for (int k = 1; k <= e; k++) {
            ph = (k == 1) ? p - 1 : ph * p;
            pk *= p;
            for (int i = 0; i < sz; i++) res.push_back({res[i].first * pk, res[i].second *
ph});
        }
    }
    return res;
}

ll necklaces(ll n, ll k) {                      // (1/n) sum_{d|n} phi(d) k^{n/d}
    ll s = 0;
    for (auto [d, ph] : divisorsWithPhi(n)) s = (s + ph % MOD * pw(k, n / d)) % MOD;
    return s * inv(n % MOD) % MOD;
}
ll bracelets(ll n, ll k) {
    ll refl = (n % 2) ? n % MOD * pw(k, (n + 1) / 2) % MOD
                      : (n / 2) % MOD * ((pw(k, n / 2) + pw(k, n / 2 + 1)) % MOD) % MOD;
    return (necklaces(n, k) * (n % MOD) + refl) % MOD * inv(2 * n % MOD) % MOD;
}
ll properNecklaces(ll n, ll k) {                // adjacent beads must differ
    ll s = 0;
```

```
    for (auto [d, ph] : divisorsWithPhi(n)) {    // rotation with n/d cycles: there are
phi(d) of them
        ll c = n / d;                            // cycle structure of the quotient: C_c
        ll fix = (pw(k - 1, c) + (c % 2 ? MOD - (k - 1) % MOD : (k - 1) % MOD)) % MOD;
        s = (s + ph % MOD * fix) % MOD;
    }
    return s * inv(n % MOD) % MOD;
}

int main() {
    ll n, k;
    cin >> n >> k;
    cout << necklaces(n, k) << ' ' << bracelets(n, k) << ' ' << properNecklaces(n, k) <<
'\n';
}
```

All three functions agree with explicit orbit enumeration for every n ≤ 8, k ≤ 4.

6.2 Sublinear sums: π(n), Σφ by min-25, Σφ and Mertens by Du

Input: n (≤ 10^{11}). Output: π(n), Σ_{i≤n} φ(i) mod 10^9+7 (computed twice, by min-25 and by Du), and M(n).

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 1'000'000'007;

ll isqrt(ll n) { ll s = sqrtl(n); while (s * s > n) s--; while ((s + 1) * (s + 1) <= n)
s++; return s; }

struct Lucy {                                    // pi(v) for all v in {n / i}
    ll n, sq; vector<ll> vals, cnt;
    int idx(ll v) const { return v <= sq ? (int)vals.size() - (int)v : (int)(n / v) - 1; }
    Lucy(ll n_) : n(n_), sq(isqrt(n_)) {
        for (ll i = 1; i <= n; i = n / (n / i) + 1) vals.push_back(n / i);
        for (ll v : vals) cnt.push_back(v - 1);
        for (ll p = 2; p <= sq; p++) {
            if (cnt[idx(p)] == cnt[idx(p - 1)]) continue;
            ll before = cnt[idx(p - 1)];
            for (size_t i = 0; i < vals.size() && vals[i] >= p * p; i++)
                cnt[i] -= cnt[idx(vals[i] / p)] - before;
        }
    }
    ll pi(ll v) const { return cnt[idx(v)]; }
};

struct Min25Phi {                                // sum_{i<=n} phi(i) mod MOD
    ll n, sq; vector<ll> vals, g0, g1, primes, psum;
    int idx(ll v) const { return v <= sq ? (int)vals.size() - (int)v : (int)(n / v) - 1; }
    Min25Phi(ll n_) : n(n_), sq(isqrt(n_)) {
        vector<char> comp(sq + 2, 0);
```

```
        for (ll i = 2; i <= sq; i++) if (!comp[i]) { primes.push_back(i); for (ll j = i *
i; j <= sq; j += i) comp[j] = 1; }
        psum.assign(primes.size() + 1, 0);
        for (size_t k = 0; k < primes.size(); k++) psum[k + 1] = (psum[k] + primes[k]) %
MOD;
        const ll inv2 = (MOD + 1) / 2;
        for (ll i = 1; i <= n; i = n / (n / i) + 1) {
            ll v = n / i; vals.push_back(v);
            g0.push_back((v - 1) % MOD);                                        // # of
2..v
            g1.push_back((v % MOD * ((v + 1) % MOD) % MOD * inv2 % MOD - 1 + MOD) % MOD);
// sum of 2..v
        }
        for (size_t k = 0; k < primes.size(); k++) {
            ll p = primes[k], c0 = k, c1 = psum[k];
            for (size_t i = 0; i < vals.size() && vals[i] >= p * p; i++) {
                int j = idx(vals[i] / p);
                g0[i] = ((g0[i] - (g0[j] - c0)) % MOD + MOD) % MOD;
                g1[i] = ((g1[i] - p % MOD * ((g1[j] - c1 + MOD) % MOD)) % MOD + MOD) %
MOD;
            }
        }
    }
    ll S(ll x, size_t k) const {                // phi-sum over 2..x with smallest prime
factor >= primes[k]
        ll pk = k < primes.size() ? primes[k] : sq + 1;
        if (x < pk) return 0;
        int id = idx(x);
        ll res = ((g1[id] - g0[id] - (psum[k] - (ll)k)) % MOD + 2 * MOD) % MOD;   //
phi(p) = p - 1
        for (size_t t = k; t < primes.size() && primes[t] * primes[t] <= x; t++) {
            ll p = primes[t], pe = p, f = p - 1;                                // f =
phi(p^e)
            for (; pe * p <= x; pe *= p, f *= p)
                res = (res + f % MOD * S(x / pe, t + 1) + (f * p) % MOD) % MOD;
        }
        return res;
    }
    ll sumPhi() const { return (S(n, 0) + 1) % MOD; }
};

struct Du {                                     // Phi(n) mod MOD and Mertens M(n),
O(n^{2/3})
    ll n; int lim; vector<ll> sphi, smu; unordered_map<ll, ll> mp, mm;
    Du(ll n_) : n(n_) {
        lim = max(10.0, pow((double)n, 2.0 / 3));
        vector<int> ph(lim + 1), mu(lim + 1), pr; vector<char> comp(lim + 1, 0);
        ph[1] = mu[1] = 1;
        for (int i = 2; i <= lim; i++) {        // linear sieve
            if (!comp[i]) { pr.push_back(i); ph[i] = i - 1; mu[i] = -1; }
            for (int p : pr) {
                if ((ll)p * i > lim) break;
                comp[p * i] = 1;
                if (i % p == 0) { ph[p * i] = ph[i] * p; mu[p * i] = 0; break; }
                ph[p * i] = ph[i] * (p - 1); mu[p * i] = -mu[i];
```

```
            }
        }
        sphi.assign(lim + 1, 0); smu.assign(lim + 1, 0);
        for (int i = 1; i <= lim; i++) sphi[i] = (sphi[i - 1] + ph[i]) % MOD, smu[i] =
smu[i - 1] + mu[i];
    }
    ll Phi(ll x) {
        if (x <= lim) return sphi[x];
        auto it = mp.find(x); if (it != mp.end()) return it->second;
        ll r = (ll)((__int128)x * (x + 1) / 2 % MOD);
        for (ll l = 2, rr; l <= x; l = rr + 1) {
            rr = x / (x / l);
            r = (r - (rr - l + 1) % MOD * Phi(x / l)) % MOD;
        }
        return mp[x] = (r + MOD) % MOD;
    }
    ll Mertens(ll x) {
        if (x <= lim) return smu[x];
        auto it = mm.find(x); if (it != mm.end()) return it->second;
        ll r = 1;
        for (ll l = 2, rr; l <= x; l = rr + 1) { rr = x / (x / l); r -= (rr - l + 1) *
Mertens(x / l); }
        return mm[x] = r;
    }
};

int main() {
    ll n;
    cin >> n;
    Du du(n);
    cout << Lucy(n).pi(n) << ' ' << Min25Phi(n).sumPhi() << ' ' << du.Phi(n) << ' ' <<
du.Mertens(n) << '\n';
}
```

Validation: for 300 values of n up to $2 \cdot 10^6$ (including every $n \le 100$), $\pi(v)$ for all $v \in V$, $\Sigma\varphi$ (both methods) and $M(n)$ match a linear sieve. For $n = 10^{10}$: $\pi = 455052511$, min-25 ≈ 0.3 s, Du ≈ 0.3 s.

6.3 Partition numbers via the pentagonal theorem

```
vector<ll> partitions(int n, ll mod) {          // p(0..n), O(n sqrt n)
    vector<ll> p(n + 1, 0); p[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int k = 1;; k++) {
            int g1 = k * (3 * k - 1) / 2, g2 = k * (3 * k + 1) / 2;
            if (g1 > i) break;
            ll s = (p[i - g1] + (g2 <= i ? p[i - g2] : 0)) % mod;
            p[i] = (p[i] + (k & 1 ? s : mod - s)) % mod;
        }
    return p;
}
```

7. Common mistakes

1. **Dividing by |G| when the modulus is not prime or divides |G|.** Look for algebraic cancellation (POJ 2154) or compute modulo P·|G| and divide exactly.
2. **Wrong reflection cases for bracelets.** Odd n: all axes through a bead. Even n: half through two beads, half through none.
3. **Applying Burnside to a set that is not closed under the group.** If the constraint is not rotation-invariant (e.g. "bead 0 is red"), Burnside does not apply directly.
4. **Floor blocks with two variables**: r must be min(n/(n/l), m/(m/l)), and the loop must stop at min(n, m).
5. **Overflow in n(n+1)/2 for n ≈ 10^{10}–10^{11}.** Either reduce both factors modulo MOD and multiply by the inverse of 2, or use `__int128`.
6. **Floating `sqrt` for √n.** `sqrt(1e10)` may be off by one after rounding; always correct with integer loops.
7. **min-25: missing the prime-power term** f(p^{e+1}), or iterating e while p^e ≤ x instead of p^{e+1} ≤ x (double counting).
8. **Du's sieve with too small a precomputation bound**: cost degrades towards O(n^{3/4}); with a hash map on ~10^5 keys, also use `reserve`.
9. **Confusing OGF and EGF** when combining labelled and unlabelled parts: labelled products need the binomial (EGF) convolution.

8. Practice problems

Problem	Technique
CSES 2209 — Counting Necklaces	Burnside, rotations
CSES 2210 — Counting Grids	Burnside, grid rotations
POJ 2409 — Let it Bead	bracelets
POJ 2154 — Color	necklaces with n ≤ 10^9, non-prime modulus
UVa 10601 — Cubes	Pólya with colour multiplicities on cube edges
SPOJ VLATTICE — Visible Lattice Points	Möbius in 3D, floor blocks
CF 803F — Coprime Subsequences	Möbius over a multiset
CF 547C — Mike and Foam	dynamic Möbius counting
CF 1559E — Mocha and Stars	Möbius + bounded knapsack
CF 438E — The Child and Binary Tree	GF equation, polynomial sqrt and inverse
Library Checker — Counting Primes	Lucy, n ≤ 10^{11}
Library Checker — Sum of Totient Function	Du's sieve or min-25
CF 665F — Four Divisors	Lucy π table
LOJ 6053 — 简单的函数	min-25 with irregular f(p^e)

9. Further reading

- R. Graham, D. Knuth, O. Patashnik, *Concrete Mathematics*, ch. 4 (number theory) and ch. 7 (generating functions).
- P. Flajolet, R. Sedgewick, *Analytic Combinatorics*, Part A — the symbolic method (the construction table in §4.1).
- H. Wilf, *generatingfunctionology* (free online).
- cp-algorithms: "Burnside's lemma / Pólya enumeration theorem", "Möbius function" — https://cp-algorithms.com/
- Codeforces blog by Nisiyama_Suzune, "[Tutorial] Math note — Möbius inversion" (2018), and the Codeforces blogs on "min_25 sieve" and "Lucy_Hedgehog's prime counting" (originally a Project Euler forum post for problem 10).